CyberAudit

Custom User Model

UUID primary key · Email login · RBAC (client / admin) · JWT auth

Project / Projet	CyberAudit Platform
Module	apps/accounts/
Files	models.py · serializers.py · views.py · tests.py
Auth strategy	JWT (SimpleJWT) access + refresh tokens, blacklist on logout
Primary key	UUID4 prevents IDOR attacks (sequential ID enumeration)
Login field	email (USERNAME_FIELD), no username field
Roles	client (default) | admin, TextChoices, stored in DB
Tests	14 pytest tests across 5 classes, 100% endpoint coverage
Date	June 2026 / Juin 2026

1. Architecture Overview

1. Vue d'ensemble de l'architecture

The accounts app is CyberAudit's authentication backbone. It replaces Django's built-in auth.User with a custom model that uses email as the login credential and UUID as the primary key. Role-based access control (RBAC) is enforced at the view layer using the role field.

L'app accounts est le backbone d'authentification de CyberAudit. Elle remplace le auth.User integre de Django par un modele personnalise qui utilise l'email comme identifiant de connexion et l'UUID comme cle primaire. Le controle d'acces base sur les roles (RBAC) est applique au niveau des vues via le champ role.

Module structure / Structure du module

models.py	User model + UserManager. Core data layer. Modele User + UserManager. Couche de donnees principale.
serializers.py	UserSerializer (read), RegisterSerializer (create), ChangePasswordSerializer (update). UserSerializer (lecture), RegisterSerializer (creation), ChangePasswordSerializer (mise a jour).
views.py	8 API endpoints (register, login, logout, token/refresh, me GET/PATCH/DELETE, change-password). 8 endpoints API (register, login, logout, token/refresh, me GET/PATCH/DELETE, change-password).
tests.py	14 pytest tests, 5 classes covering model + all endpoints. 14 tests pytest, 5 classes couvrant le modele + tous les endpoints.
urls.py	URL routing, maps paths to APIView classes. Routage d'URL mappe les chemins aux classes APIView.

Design decisions / Decisions de conception

Django recommends substituting the user model at the start of a project because changing it later requires recreating all migration history. CyberAudit uses AbstractBaseUser (not AbstractUser) which gives full control over all fields, there is no username field at all.

Django recommande de substituer le modele utilisateur au debut d'un projet car le changer ensuite necessite de recreer tout l'historique des migrations. CyberAudit utilise AbstractBaseUser (pas AbstractUser) ce qui donne un controle total sur tous les champs, il n'y a aucun champ username.

2. User Model : models.py

2. Modele User : models.py

Class inheritance / Heritage de classe

```
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager,
PermissionsMixin

class User(AbstractBaseUser, PermissionsMixin):
    # AbstractBaseUser  → password hashing, last_login, has_perm()
    # PermissionsMixin  → is_superuser, groups, user_permissions (Django admin
compat)
```

Fields overview / Vue d'ensemble des champs

Field	Type	Constraint	Purpose EN / FR
id	UUIDField	PK, auto	UUID4, editable=False. Prevents IDOR attacks. UUID4, editable=False. Previent les attaques IDOR.
email	EmailField	unique=True	LOGIN CREDENTIAL USERNAME_FIELD = 'email'. IDENTIFIANT USERNAME_FIELD = 'email'.
first_name	CharField	max_length=50	Given name. Required on registration. Prenom. Requis a l'inscription.
last_name	CharField	max_length=50	Family name. Required on registration. Nom de famille. Requis a l'inscription.
company_name	CharField	max_length=100, blank=True	Optional SME company name. Nom de l'entreprise PME (optionnel).
role	CharField	choices=Role, default='client'	RBAC field: 'client' | 'admin'. Checked in views. Champ RBAC : 'client' | 'admin'. Verifie dans les vues.
is_active	BooleanField	default=True	Soft-delete flag. False = account disabled. Flag de suppression douce. False = compte desactive.

is_staff	BooleanField	default=False	Django admin access. Set to True for admin role. Acces a l'admin Django. True pour le role admin.
created_at	DateTimeField	auto_now_add=True	Immutable creation timestamp. Horodatage de creation immuable.

Role.TextChoices : RBAC

```
class Role(models.TextChoices):
    CLIENT = 'client', 'Client'
    ADMIN  = 'admin',  'Administrateur'

# Stored as string in DB — readable in queries and Django admin
# Default: Role.CLIENT — all registrations are clients by default
# Promotion to admin: done manually in Django admin or create_superuser
```

UUID primary key : IDOR prevention / Prevention IDOR

Sequential integer PKs (1, 2, 3...) allow attackers to enumerate resources by guessing adjacent IDs. UUID4 generates a random 128-bit value: there are 2^122 possible values, making enumeration practically impossible.

Les PK entiers séquentiels (1, 2, 3...) permettent aux attaquants d'énumérer les ressources en devinant les IDs adjacents. UUID4 génère une valeur aléatoire de 128 bits : il y a 2^122 valeurs possibles, rendant l'énumération pratiquement impossible.

```
id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
# editable=False → cannot be changed via forms or serializers
# default=uuid.uuid4 → auto-generated, no argument needed on User.objects.create()
# Stored as a 36-char string: '550e8400-e29b-41d4-a716-446655440000'
```

full_name property / Propriete full_name

```
@property
def full_name(self):
    return f'{self.first_name} {self.last_name}'.strip()
    # .strip() handles the case where one name is empty
    # Read-only — not a DB field, computed on the fly
```

Meta class

```
class Meta:
    db_table = 'users'      # explicit table name (not 'accounts_user')
```

```
    ordering = ['-created_at']  # newest first in all querysets

USERNAME_FIELD  = 'email'     # used by authenticate(), login(), JWT
REQUIRED_FIELDS = []          # createsuperuser only asks email + password
```

3. UserManager : BaseUserManager

3. UserManager : BaseUserManager

Django requires a custom Manager when using AbstractBaseUser. UserManager overrides create_user() and create_superuser() to handle email normalisation and password hashing.
Django exige un Manager personnalise lors de l'utilisation d'AbstractBaseUser. UserManager remplace create_user() et create_superuser() pour gerer la normalisation de l'email et le hachage du mot de passe.

create_user()

```
def create_user(self, email, password=None, **extra_fields):
    if not email:
        raise ValueError("L'adresse email est obligatoire.")
    email = self.normalize_email(email)  # lowercases domain: User@EXAMPLE.COM →
User@example.com
    user  = self.model(email=email, **extra_fields)
    user.set_password(password)          # hashes using Django's password hasher
(PBKDF2 by default)
    user.save(using=self._db)
    return user
```

create_superuser()

```
def create_superuser(self, email, password=None, **extra_fields):
    extra_fields.setdefault('is_staff',      True)       # Django admin access
    extra_fields.setdefault('is_superuser',  True)       # all permissions
    extra_fields.setdefault('role',          User.Role.ADMIN)  # role='admin'
    return self.create_user(email, password, **extra_fields)
    # Used by: python manage.py createsuperuser
```

normalize_email() : detail

User@EXAMPLE.COM	User@example.com
alice@test.com	alice@test.com
ALICE@GMAIL.COM	ALICE@gmail.com

4. Serializers : serializers.py

4. Serialiseurs : serializers.py

UserSerializer : read / lecture

Used for GET /me/ responses and embedded in login/register token responses. The password is never included. id, email, role, is_active, created_at are read_only, they cannot be updated via PATCH.

Utilise pour les reponses GET /me/ et integre dans les reponses de token login/register. Le mot de passe n'est jamais inclus. id, email, role, is_active, created_at sont en lecture seule, ils ne peuvent pas etre mis a jour via PATCH.

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields =
['id','email','first_name','last_name','company_name','role','is_active','created_at']
        read_only_fields = ['id','email','role','is_active','created_at']
        # PATCH /me/ can only update: first_name, last_name, company_name
```

RegisterSerializer : create / creation

Handles new account creation. password is write_only (never returned in response), min_length=8, and validated by Django's validate_password() which checks for common passwords and minimum complexity. create() delegates to User.objects.create_user() which hashes the password.

Gere la creation de nouveaux comptes. password est write_only (jamais retourne en reponse), min_length=8, et valide par validate_password() de Django qui verifie les mots de passe courants et la complexite minimale. create() delegue a User.objects.create_user() qui hache le mot de passe.

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(
        write_only=True,
        min_length=8,
        validators=[validate_password],   # Django's built-in password validator
        style={'input_type': 'password'}, # hides in Browsable API
    )
    class Meta:
        model = User
        fields = ['email','password','first_name','last_name','company_name']

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)  # handles hashing
```

ChangePasswordSerializer : update / mise à jour

```
class ChangePasswordSerializer(serializers.Serializer):  # NOT ModelSerializer
    old_password = serializers.CharField(write_only=True)
    new_password = serializers.CharField(write_only=True, min_length=8,
validators=[validate_password])

    def validate_old_password(self, value):
        user = self.context['request'].user  # accessed via context, not
request.data
        if not user.check_password(value):   # Django's constant-time comparison
            raise serializers.ValidationError('Mot de passe actuel incorrect.')
        return value
```

5. API Endpoints : views.py

5. Endpoints API : views.py

Endpoint	Method	Auth	Description EN / FR
/api/auth/register/	POST	None	Create account + return JWT tokens. Creer un compte + retourner les tokens JWT.
/api/auth/login/	POST	None	Validate credentials + return JWT tokens. Valider les identifiants + retourner les tokens JWT.
/api/auth/logout/	POST	JWT	Blacklist refresh token to invalidate session. Blacklister le refresh token pour invalider la session.
/api/auth/token/refresh/	POST	None	SimpleJWT native view, exchange refresh for new access. Vue native SimpleJWT, échanger refresh contre un nouvel access.
/api/auth/me/	GET	JWT	Return authenticated user profile. Retourner le profil de l'utilisateur connecte.
/api/auth/me/	PATCH	JWT	Partial update of first_name / last_name / company_name. Mise a jour partielle de first_name / last_name / company_name.
/api/auth/me/	DELETE	JWT	Soft-delete: sets is_active=False, preserves DB row. Suppression douce : passe is_active=False, conserve la ligne DB.
/api/auth/change-password/	POST	JWT	Verify old password, hash and save new password. Vérifier l'ancien mot de passe, hacher et enregistrer le nouveau.

_token_response() helper : shared by register + login

```
def _token_response(user):
    refresh = RefreshToken.for_user(user)  # generates refresh + embeds access
    return {
        'access':  str(refresh.access_token),   # short-lived (5 min default)
        'refresh': str(refresh),                 # long-lived (7 days default)
        'user':    UserSerializer(user).data,    # profile (no password)
    }
# Called by both RegisterView.post() and LoginView.post()
# Frontend stores access in memory, refresh in localStorage
```

LoginView : anti-enumeration security / Sécurité anti-enumeration

LoginView uses the same error message whether the email is unknown or the password is wrong: 'Email ou mot de passe incorrect.' This prevents attackers from discovering which email addresses are registered (account enumeration attack).

LoginView utilise le meme message d'erreur que l'email soit inconnu ou que le mot de passe soit faux : 'Email ou mot de passe incorrect.' Cela empeche les attaquants de decouvrir quelles adresses email sont enregistrees (attaque d'enumeration de comptes).

```
try:
    user = User.objects.get(email=email)
except User.DoesNotExist:
    return Response({'non_field_errors': ['Email ou mot de passe incorrect.']},
                    status=status.HTTP_401_UNAUTHORIZED)

if not user.check_password(password):
    return Response({'non_field_errors': ['Email ou mot de passe incorrect.']},
                    status=status.HTTP_401_UNAUTHORIZED)
# Same message for both cases → attacker cannot determine if email exists
```

LogoutView : JWT blacklisting / Blacklistage JWT

Standard JWT tokens are stateless, once issued they are valid until expiry. SimpleJWT's blacklist feature stores invalidated tokens in a DB table (outstanding_token, blacklisted_token) so that a logged-out refresh token cannot be reused.

Les tokens JWT standard sont sans étatn une fois emis, ils sont valides jusqu'a expiration. La fonctionnalite de blacklist de SimpleJWT stocke les tokens invalides dans une table DB (outstanding_token, blacklisted_token) afin qu'un refresh token deconnecte ne puisse pas être réutilisé.

```
def post(self, request):
    refresh_token = request.data.get('refresh')
    if not refresh_token:
        return Response({'detail': "Le champ 'refresh' est requis."}, status=400)
    try:
        token = RefreshToken(refresh_token)
        token.blacklist()  # adds to blacklisted_token table
```

```
        except TokenError:
            return Response({'detail': 'Token invalide ou deja revoque.'}, status=400)
        return Response(status=status.HTTP_204_NO_CONTENT)
```

MeView DELETE : soft delete / Suppression douce

```
def delete(self, request):
    request.user.is_active = False
    request.user.save(update_fields=['is_active'])  # only updates this column
    return Response(status=status.HTTP_204_NO_CONTENT)
    # Row is preserved in DB — audit trail intact
    # LoginView checks is_active and returns 403 if False
    # Admin can reactivate by setting is_active=True in Django admin
```

6. Tests : accounts/tests.py

6. Tests : accounts/tests.py

14 pytest tests are organised into 5 classes. Each class targets a specific component: the model itself, or one of the four main endpoint groups. All tests are marked @pytest.mark.django_db and use fixtures (api_client, auth_client, client_user, make_client_user, make_admin_user) defined in conftest.py.

14 tests pytest sont organisés en 5 classes. Chaque classe cible un composant specifique : le modele lui-meme, ou l'un des quatre groupes d'endpoints principaux. Tous les tests sont marques @pytest.mark.django_db et utilisent des fixtures (api_client, auth_client, client_user, make_client_user, make_admin_user) definies dans conftest.py.

Test class	Count	What is tested EN / FR
TestUserModel	8	Password hashing, default role, __str__, full_name, UUID PK length, admin flags, empty email error, create_superuser flags. Hachage du mot de passe, role par defaut, __str__, full_name, longueur UUID, flags admin, erreur email vide, flags create_superuser.
TestRegisterView	3	201 + tokens on success, 400 on duplicate email, 400 on missing email. 201 + tokens en succes, 400 sur email duplique, 400 sur email manquant.
TestLoginView	3	200 + tokens on success, 401 on wrong password, 403 on inactive account. 200 + tokens en succes, 401 mauvais mot de passe, 403 compte inactif.
TestMeView	3	200 profile when authenticated, 401 when unauthenticated, 200 + updated first_name on PATCH. 200 profil si authentifie, 401 non authentifie, 200 + first_name mis a jour sur PATCH.
TestLogoutView	2	204 + blacklist on success, 400 when refresh field missing. 204 + blacklist en succes, 400 si champ refresh manquant.
TestChangePasswordView	2	200 on success with correct old password, 400 when old password is wrong. 200 en succès avec ancien mot de passe correct, 400 si ancien mot de passe faux.

Key test: UUID primary key

```
def test_uuid_primary_key(self, make_client_user):
    user = make_client_user()
    assert len(str(user.id)) == 36  # '550e8400-e29b-41d4-a716-446655440000'
    # Format: 8-4-4-4-12 hex digits separated by hyphens
```

Key test: anti-enumeration (same error both cases)

```
def test_login_wrong_password_returns_401(self, api_client, client_user):
    resp = api_client.post('/api/auth/login/',
                           {'email': 'marie@example.com', 'password':
'WrongPassword!'})
    assert resp.status_code == status.HTTP_401_UNAUTHORIZED
    # Same 401 as for unknown email — attacker cannot distinguish
```

Key test: soft delete + inactive check

```
def test_login_inactive_account_returns_403(self, api_client, make_client_user):
    user = make_client_user(email='inactive@example.com')
    user.is_active = False
    user.save()
    resp = api_client.post('/api/auth/login/',
                           {'email': 'inactive@example.com', 'password':
'Secur1ty!'})
    assert resp.status_code == status.HTTP_403_FORBIDDEN
    # 403 (not 401) distinguishes disabled accounts from wrong credentials
```

7. Security Summary

7. Recapitulatif securite

Security feature	Implementation EN / FR
UUID primary key	UUIDField(default=uuid.uuid4, editable=False) prevents sequential IDOR. UUIDField(default=uuid.uuid4, editable=False) : previent les IDOR sequentiels.
Password hashing	set_password() → PBKDF2-SHA256 with salt (Django default). Never stored in plain text. set_password() → PBKDF2-SHA256 avec sel (defaut Django). Jamais stocke en clair.
Password validation	validate_password() on register + change-password: length, common passwords, numeric-only. validate_password() sur register + change-password : longueur, mots de passe courants, numerique seul.
Anti-enumeration	LoginView returns same 401 for unknown email and wrong password. LoginView retourne le meme 401 pour email inconnu et mauvais mot de passe.
JWT blacklist	LogoutView calls token.blacklist() invalidates refresh token server-side. LogoutView appelle token.blacklist() invalide le refresh token cote serveur.
Soft delete	DELETE /me/ sets is_active=False, row preserved for audit trail. Login returns 403. DELETE /me/ passe is_active=False, la ligne est conservée pour la piste d'audit. Login retourne 403.
Rate limiting	throttle_scope='login' on RegisterView and LoginView 5 req/min (configurable). throttle_scope='login' sur RegisterView et LoginView 5 req/min (configurable).
read_only_fields	id, email, role, is_active, created_at cannot be changed via PATCH /me/. id, email, role, is_active, created_at ne peuvent pas etre modifies via PATCH /me/.
Password never exposed	write_only=True on all password fields. UserSerializer has no password field. write_only=True sur tous les champs password. UserSerializer n'a pas de champ password.